Informe reflexivo – Spotter

Proyecto: Spotter – plataforma para encontrar compañeros y entrenadores para entrenar **Integrante:** Esneider Velasco Cruz **Institución:** Fundación Universitaria Los Libertadores

Aplicación desplegada: https://spotter-peach-delta.vercel.app/ **API:** https://spotter-ql7f.onrender.com (documentación en `/api/docs`) **Repositorio:** https://github.com/Esneider2406/Spotter

---

1. ¿Qué aprendí en este proyecto?

Este proyecto me permitió recorrer el ciclo completo de una aplicación web, desde el diseño hasta su publicación en la nube. Aprendí a construir una API REST con Node.js y Express, organizada por capas (rutas, middleware, utilidades y acceso a datos), y a proteger sus rutas con autenticación mediante tokens JWT y control de acceso por rol (usuario y entrenador).

También aprendí a conectar la aplicación a una base de datos PostgreSQL alojada en la nube con Neon, a crear las tablas mediante un script SQL y a manejar la conexión segura con SSL. En el frontend, trabajé con React y Vite, el enrutamiento con React Router y el consumo de la API desde un cliente centralizado.

Otro aprendizaje importante fue la gestión de secretos: comprendí por qué las credenciales deben vivir únicamente en variables de entorno, por qué el archivo `.env` nunca debe subirse al repositorio y cómo rotar una credencial que fue expuesta. Además, practiqué el flujo de trabajo con Git (ramas `main`, `develop` y `feature/*`, Pull Requests y revisión de código), las pruebas automatizadas con Jest y Supertest, la integración continua con GitHub Actions y la documentación de la API con OpenAPI y Swagger.

Finalmente, aprendí a desplegar cada parte en una plataforma distinta (base de datos en Neon, backend en Render y frontend en Vercel) y a configurar la comunicación entre ellas.

2. ¿Cuáles fueron los principales desafíos?

Error de sintaxis en el script SQL. La creación de las tablas falló por un error de sintaxis en `schema.sql`, lo que impedía iniciar la base de datos.

Errores de CORS. Fue el desafío más frecuente. El navegador bloqueaba las peticiones del frontend hacia el backend y el mensaje de error no indicaba la causa real. Se presentó de varias formas: una barra final de más en la variable `VITE_API_URL`, que generaba URLs con doble barra (`.../api/descubrir`); un valor de `CORS_ORIGIN` que no coincidía exactamente con el origen del frontend; y un puerto del entorno de desarrollo en la nube configurado como privado, que rechazaba la petición previa del navegador antes de llegar a la API.

Tokens inválidos tras cambiar el `JWT_SECRET`. Al modificar la clave de firma, los tokens guardados en el navegador dejaron de ser válidos y la API respondía con error 401, aunque el código era correcto.

Exposición de credenciales. La cadena de conexión de la base de datos, con su contraseña, quedó escrita en un archivo de ejemplo que se sube al repositorio. Esto representaba un riesgo de seguridad.

Despliegue de un repositorio con el código en subcarpetas. Render y Vercel necesitaban saber en qué carpeta estaba cada parte del proyecto, y al principio no encontraban el `package.json`.

Diferencias entre entornos. Algo que funcionaba en el entorno local fallaba en producción por variables de entorno distintas, y la primera petición al backend gratuito de Render tardaba por su estado de reposo.

3. ¿Cómo los resolvimos?

Script SQL. Revisé el mensaje de error de la base de datos, localicé la línea con el problema, corregí la sintaxis y verifiqué el resultado ejecutando `npm run db:init` hasta ver las tablas creadas en el panel de Neon.

CORS. Apliqué una estrategia de aislamiento por capas: primero comprobé que el backend respondía con `curl`, luego revisé la consola y la pestaña de red del navegador y, por último, la configuración del frontend. Así identifiqué que el problema estaba en la URL y no en la lógica de la API. Corregí las variables, hice público el puerto de desarrollo y, para que el error no se repita, modifiqué el cliente `api.js` y la configuración de CORS del backend de modo que eliminen cualquier barra final.

Tokens inválidos. Entendí que el comportamiento era el esperado: el token se firmó con la clave anterior. Lo resolví cerrando sesión (o limpiando el almacenamiento local) e iniciando sesión de nuevo, y lo documenté para evitar confusiones futuras.

Credenciales expuestas. Restablecí la contraseña de la base de datos, dejé el archivo `.env.example` únicamente con valores de ejemplo, mantuve el `.env` real fuera del repositorio mediante `.gitignore` y generé un `JWT_SECRET` largo y aleatorio.

Despliegue. Configuré el directorio raíz de cada servicio (`backend` en Render y `frontend` en Vercel), definí las variables de entorno en cada plataforma y completé el circuito indicando en el backend la URL del frontend como origen permitido. Probé el resultado final desde una ventana de incógnito.

También priorizamos el alcance: dejamos para después las funciones menos críticas y nos concentramos en que el flujo principal funcionara de extremo a extremo (registro, perfil, descubrimiento, Me gusta y Match, solicitudes y chat).

Resultados medibles

Métrica	Valor
Pruebas automatizadas (Jest) pasadas / totales	__ / __
Cobertura de código (`npm test -- --coverage`)	__ %
Casos del plan de pruebas aprobados	__ / 21
Bugs registrados / resueltos	__ / __

Conclusión

Spotter quedó desplegado y funcionando en la nube, con frontend, backend, base de datos, autenticación y documentación de la API disponibles públicamente. El mayor aprendizaje fue que la mayoría de los errores graves no estuvieron en la lógica del programa, sino en la configuración entre componentes, y que depurar por capas y documentar cada decisión ahorra mucho tiempo. Como mejora futura, planeo ampliar las pruebas automatizadas para cubrir más rutas que dependen de la base de datos y fijar la política de CORS y los secretos desde el inicio de cada proyecto.